



14

树及二叉树

Business templates allow you to enjoy flying. Business
templates allow you to enjoy flying

树

01

树的定义

02

树的存储与
遍历

03

二叉树

04

二叉树存储
与遍历



树的概念

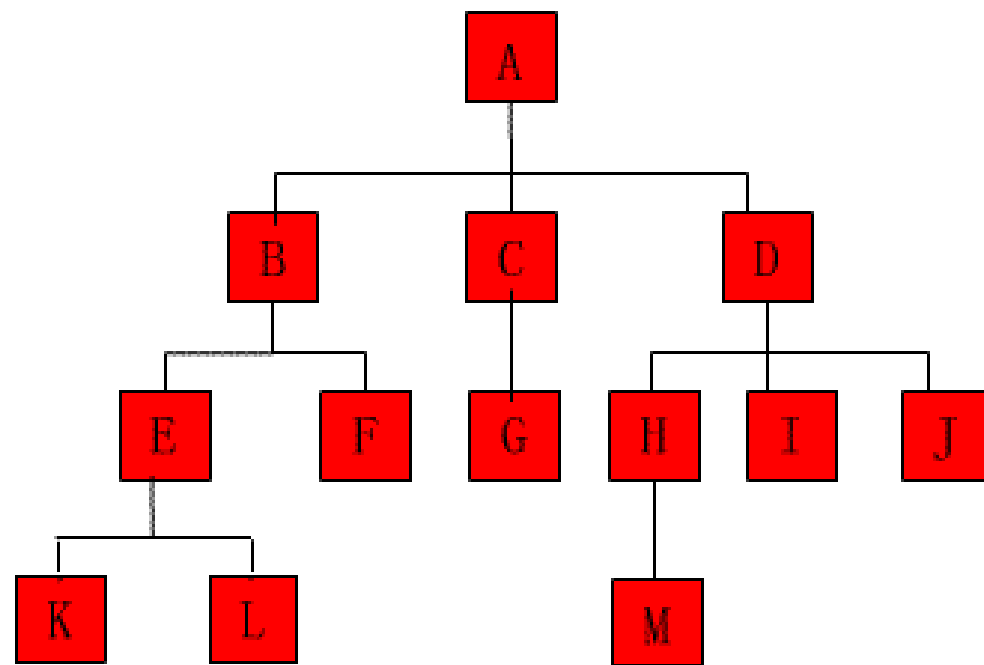


一、树的概念

1、树的定义

树是一种非线性的数据结构。

在 V 个顶点的无向图中，取其中 $V-1$ 条边，使得 V 个顶点能间接相连。得到的子图就是图的一颗生成树。





树的概念



树的递归定义如下：

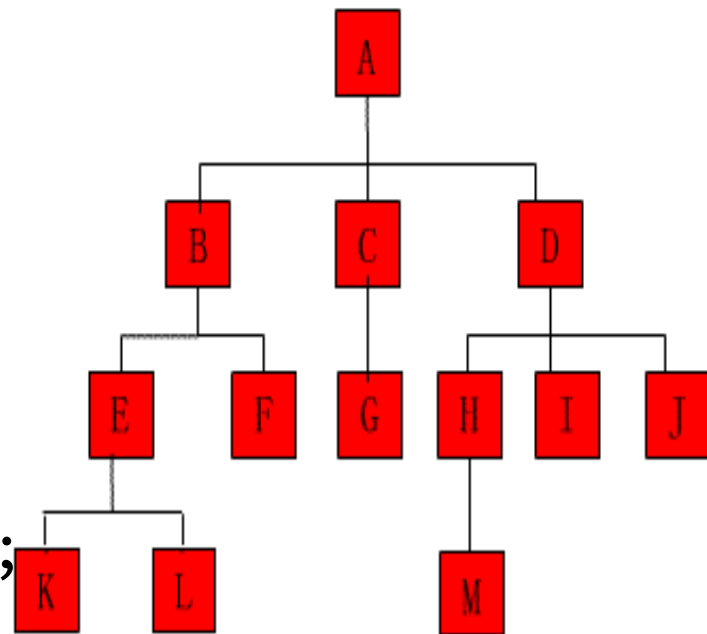
树是 $n(n>0)$ 个结点的有限集，这个集合满足以下条件：

(1)有且仅有一个结点没有前驱（父亲结点），该结点称为树的根；

(2)除根外，其余的每个结点都有且仅有一个前驱；

(3)除根外，每一个结点都通过**唯一**的路径连到根上（否则有环）。这条路径由根开始，而末端就在该结点上，且除根以外，路径上的每一个结点都是前一个结点的后继（儿子结点）；

由上述定义可知，树结构没有封闭的回路。



树形结构典型代表，家族系谱图，祖先唯一的，后代无穷的



节点的分类



2、结点的分类

结点一般分成三类

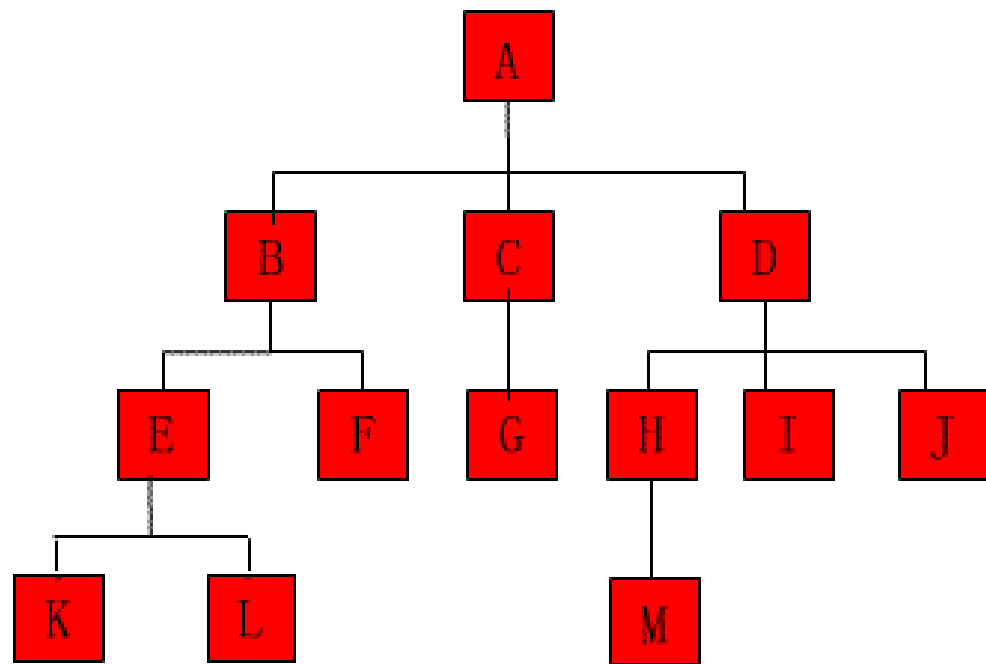
(1)根结点：没有父亲的结点。在树中有且仅有一个根结点。

(2)分支结点：除根结点外，有孩子的结点称为分支结点。

(3)叶结点：没有孩子的结点称为树叶。

根结点到每一个分支结点或叶结点的路径是**唯一**的。

从根A到结点M的唯一路径为ADHM。





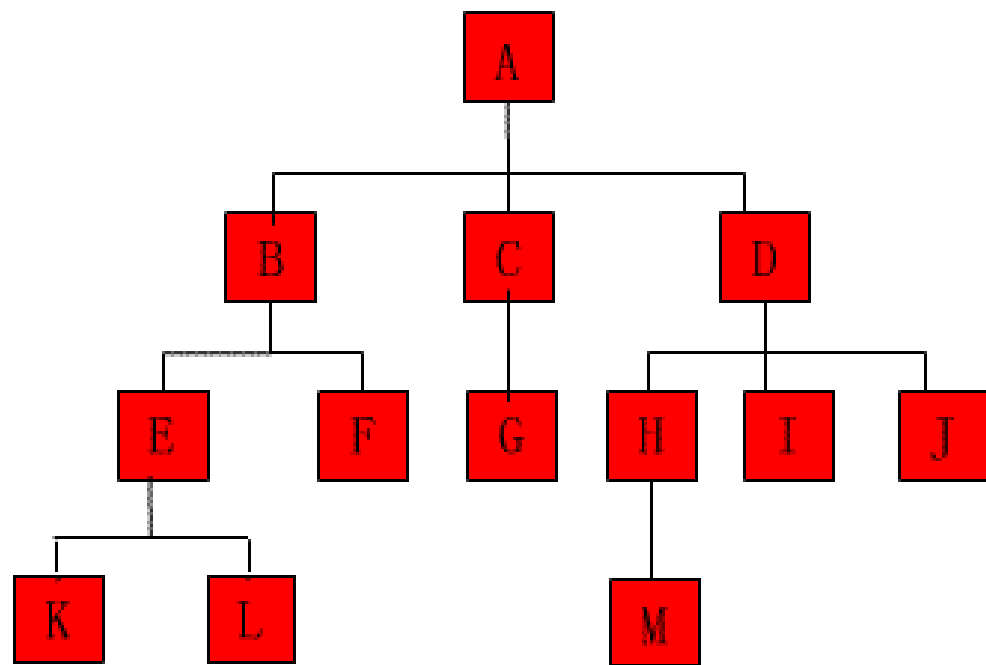
树的度



3、树的度

(1)结点的度：一个结点的**子树数目**称为该结点的度。

(2)树的度：所有结点中最大的度称为该树的度(宽度)。



树的宽度为3



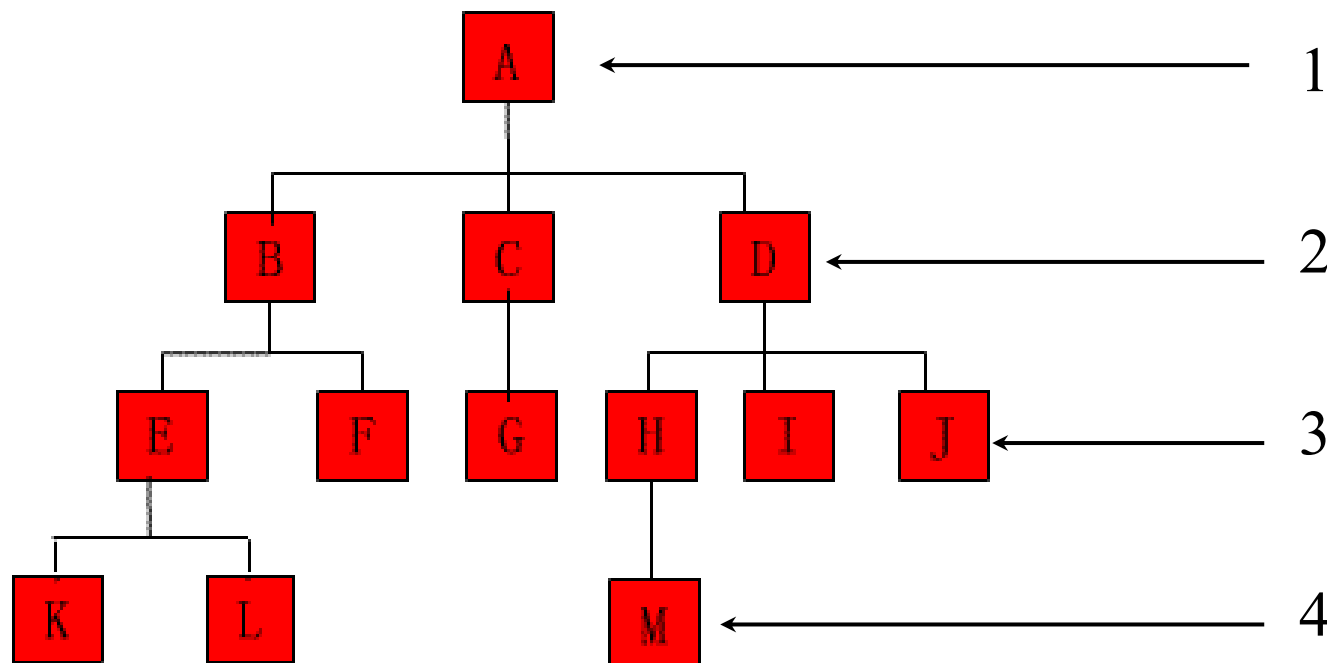
树的深度



4、树的深度（高度）

树是分层次的。结点所在的层次是从根算起的。根结点在第一层，根的儿子在第二层，其余各层依次类推。图中的树共有4层。在树中，父结点在同一层的所有结点构成兄弟关系。

树中最大的层次称为树的深度，亦称高度。图中树的深度为4。



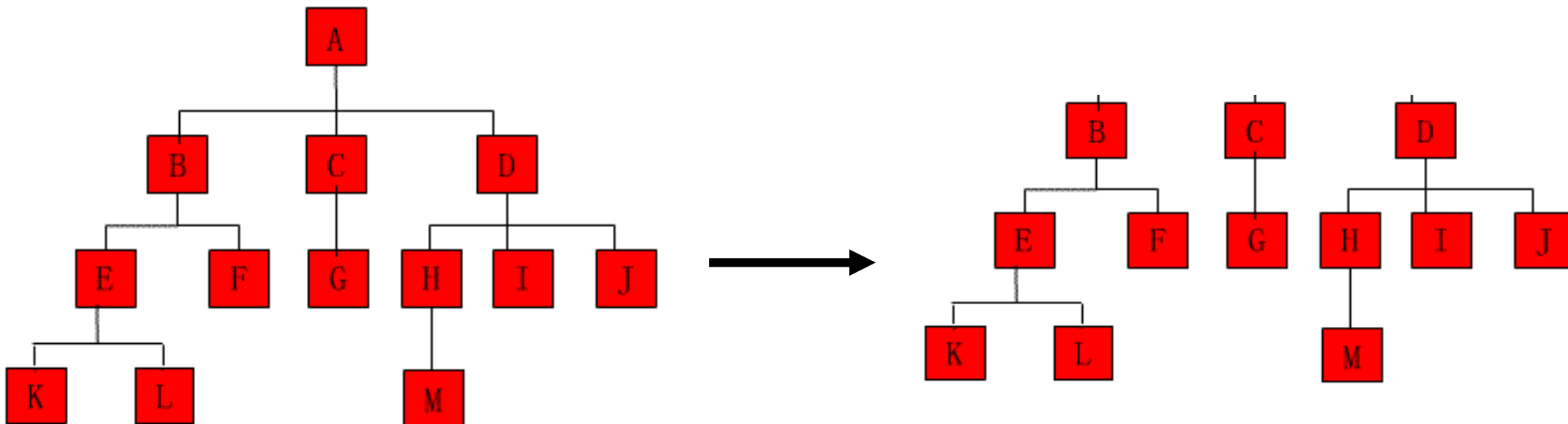


森林



5、森林

所谓森林，是指若干棵互不相交的树的集合。如图去掉根结点A，其原来的三棵子树 T_b ， T_c ， T_d 的集合 $\{T_b, T_c, T_d\}$ 就为森林，这三棵子树的具体形态如图（c）。



树的表示方法和存储

Please enter the title

02



树的表示方法



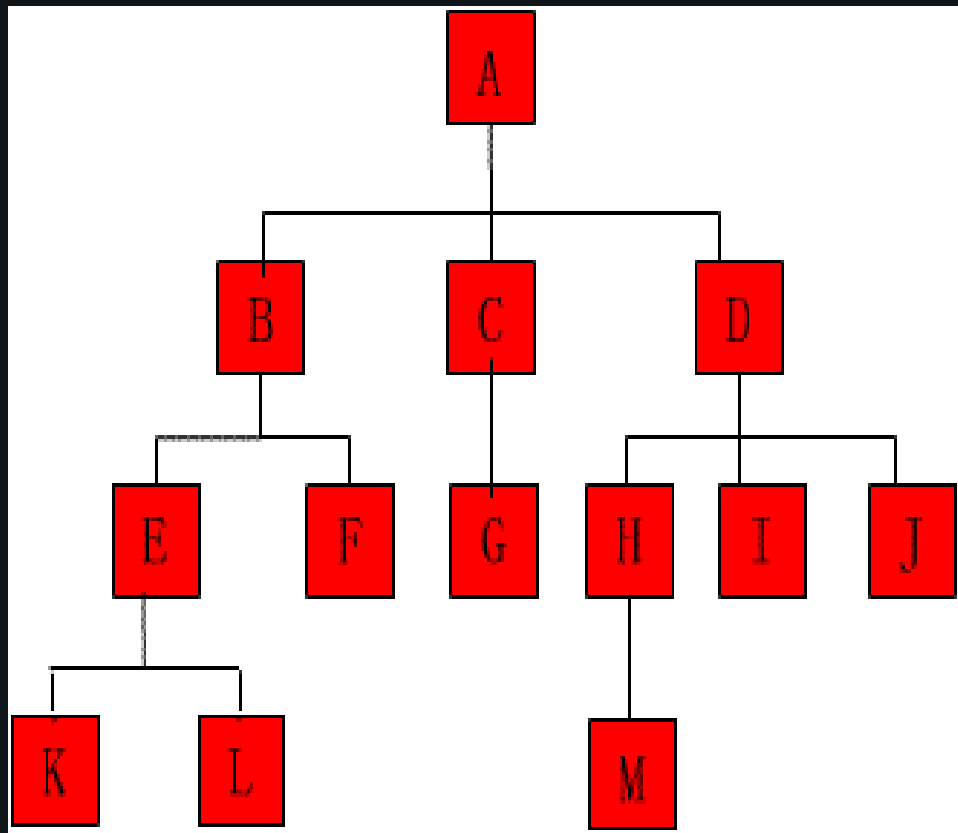
二、树的表示方法和存储结构

1、树的表示方法

树的表示方法一般有两种：

(1)自然界的树形表示法：用结点和边表示树，例如上图采用的就是自然界的树形表示法。树形表示法一般用于分析问题。

优点:直观,形象;缺点:保存困难.





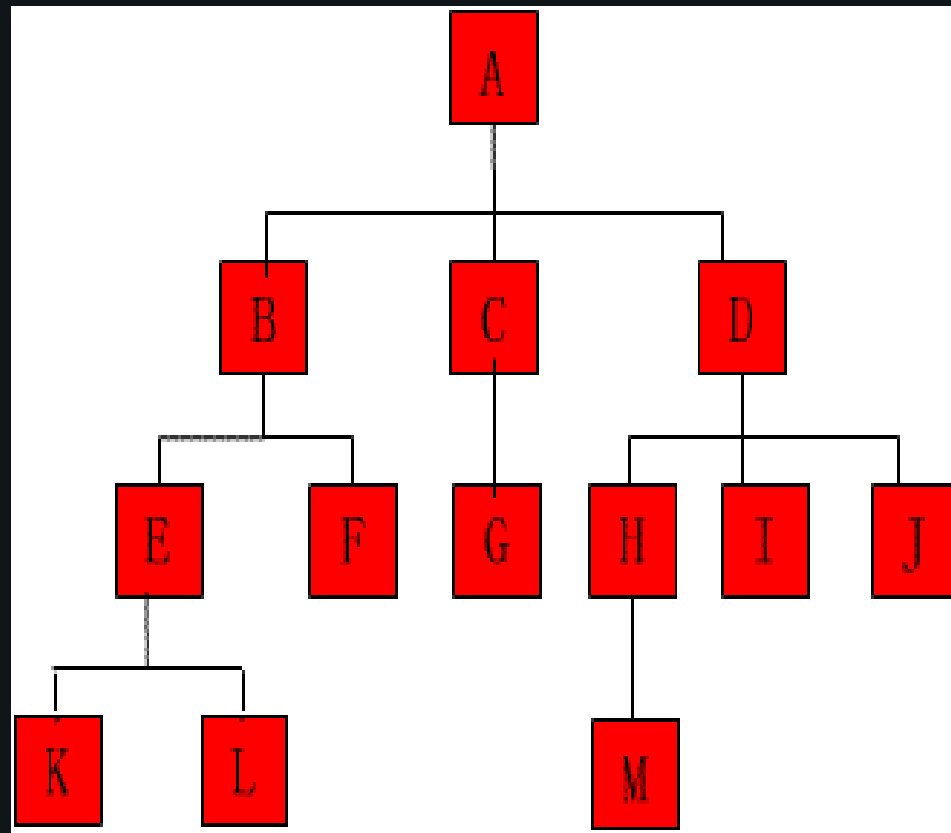
树的表示方法



(2)括号表示法:

先将根结点放入一对圆括号中，然后把它的子树按由左而右的顺序放入括号中，而对子树也采用同样方法处理：同层子树与它的根结点用圆括号括起来，同层子树之间用逗号隔开，最后用闭括号括起来。例如图可写成如下形式
(A(B(E(K,L),F),C(G),D(H(M),I,J)))

优点:易于保存;缺点:不直观.





树的存储



树的存储结构一般有三种

(1)静态的结构体数组。所有结点存储在一个数组中，数组元素为结构体类型，包括数据域和长度为 n (n 为树的度)的数组，分别存储该结点的每一个儿子的下标

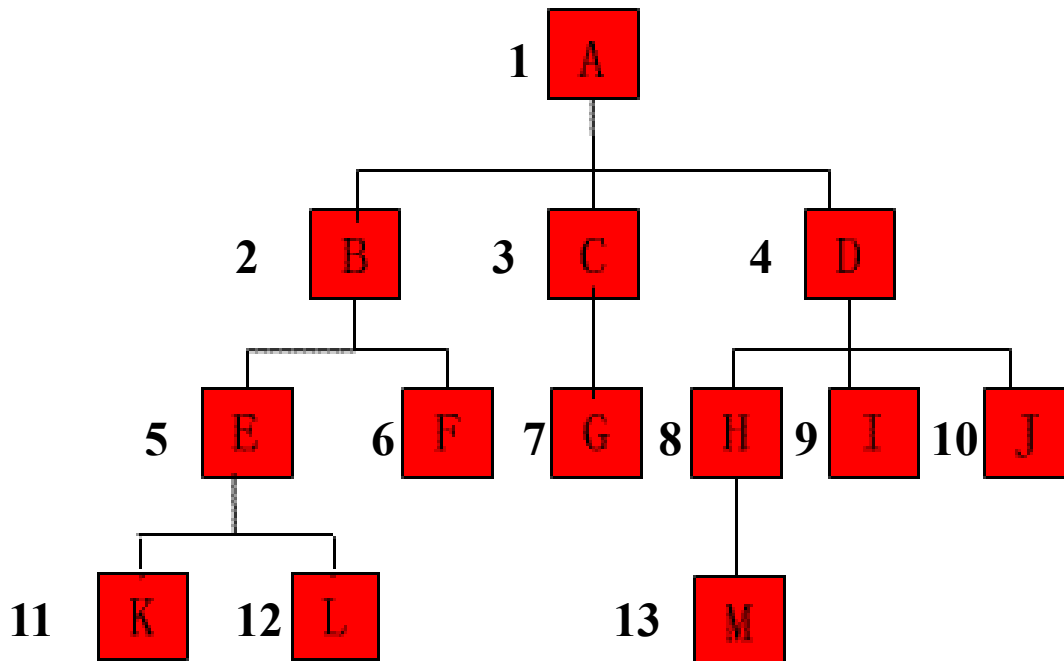
```
const int max=树的度;  
int maxn=结点个数的上限;  
struct node //结点类型  
{  
    int data; 数据域  
    int child[max+5] //指向各儿子的下标  
};  
node tree[maxn+5]; //树数组
```



树的存储



下标	信息	儿子		
1	A	2	3	4
2	B	5	6	0
3	C	7	0	0
4	D	8	9	10
5	E	11	12	0
6	F	0	0	0
7	G	0	0	0
8	H	13	0	0
9	I	0	0	0
10	J	0	0	0
11	K	0	0	0
12	L	0	0	0
13	M	0	0	0





树的存储



(2)动态的双重链表。由于树中结点可以有多个元素，所以可以用多重链表来描述比较方便。所谓多重链表，就是每个结点由数据域和 n (n 为树的度)个指针域共 $n+1$ 个域组成，其表示方法如下：

`const int maxn=树的度;`

`struct node {`

`int data;` //数据域

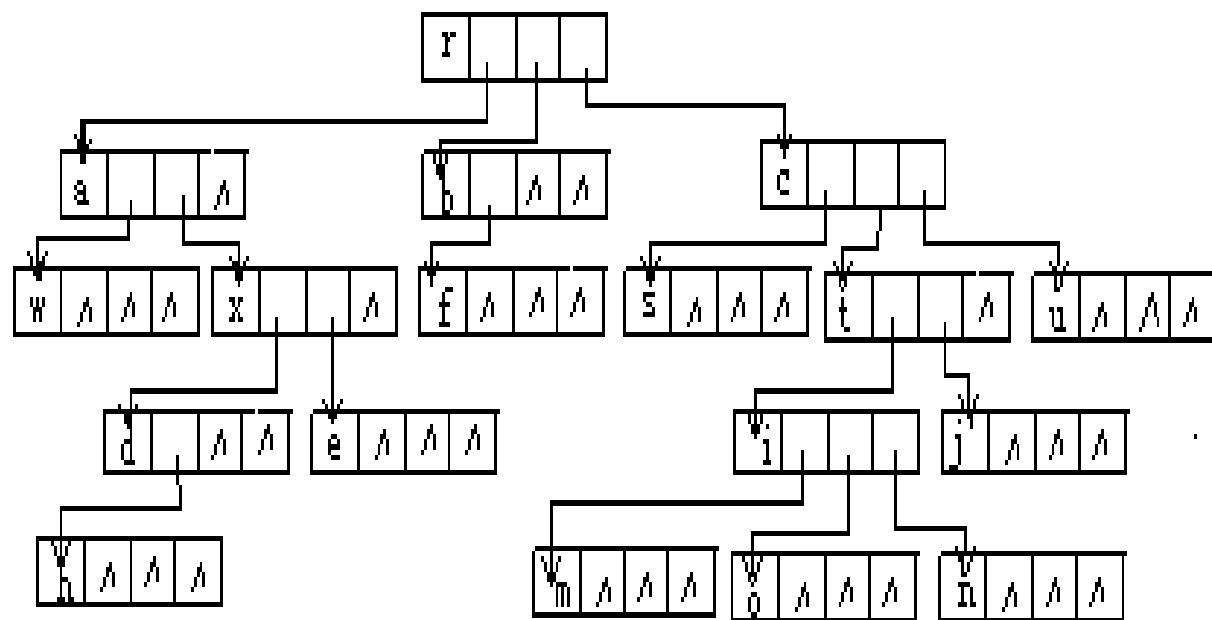
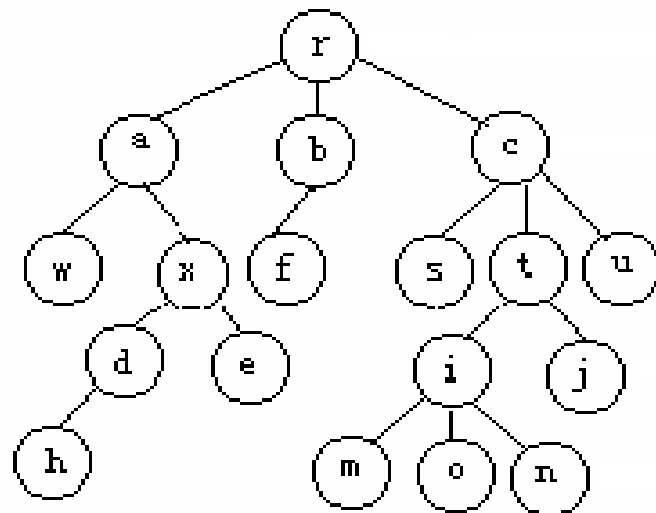
`node *child[maxn];` //指向各儿子的指针

针域

`node *father;` //指向父亲的指针

`};`

`node *root;` //根结点指针



(Λ —nil)



树的存储



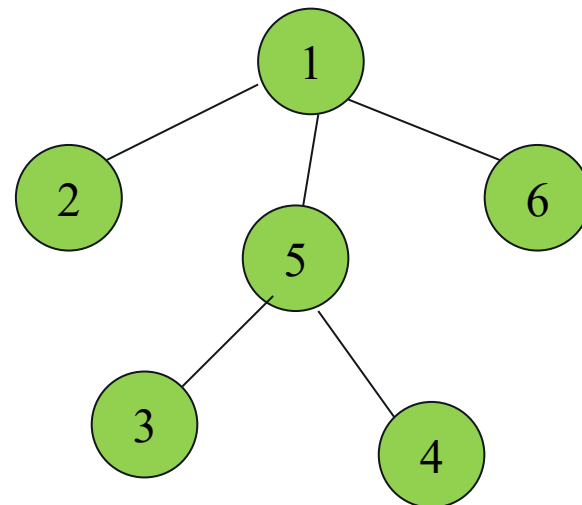
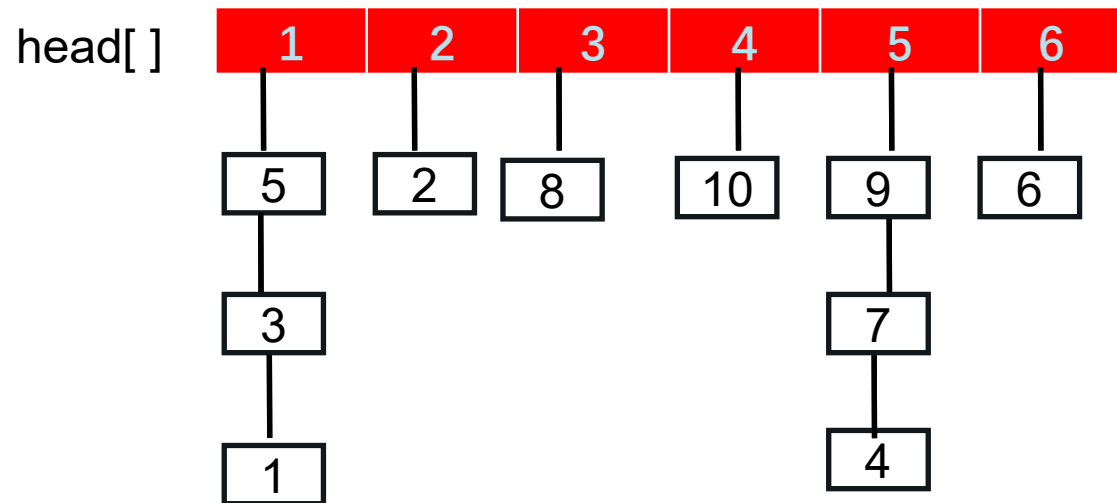
3 邻接表存储法 (点+边的方式)

下标	儿子		
1	2	5	6
2	1		
3	5		
4	5		
5	1	3	4
6	1		

```
int head[maxn];
struct node
{int to,next;
} edge[maxn*5];
void add(int u,int v)
{
    edge[++k].to=v;
    edge[k].next=head[u];
    head[u]=k;
}
```



树的存储



edge[]	1	2	3	4	5	6	7	8	9	10
	v2	v1	v5	v1	v6	v1	v3	v5	v4	v5
	0	3	1	0	3	10	4	0	7	0

1→2
1→5
1→6
5→3
5→4



树的遍历



根据对树根访问的向后顺序可以分成：

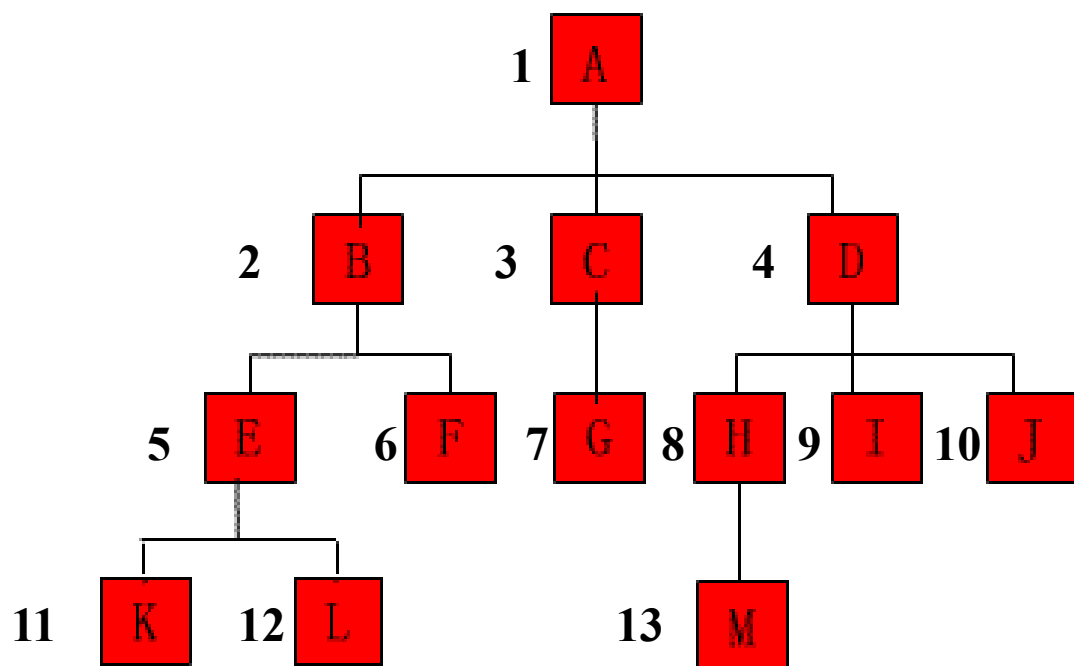
先序遍历（先根、再左子树、最后右子树）
（递归搜索算法）

后序遍历

层次遍历：按层次从小到大遍历各个子树
（广度优先搜索）



树的遍历



先序遍历: ABEKLFCDHMIJ

后序遍历: KLEFBGCMHIJDA

层次遍历: ABCDEFGHIJKLM



应用



1、树的统计

输入森林中的结点关系，统计森林中树的数量，输出树的根。

输入：

第一行：n：结点数量；k：边数；

($n, k \leq 100$)

以下k行：每行两个结点编号：i, j: i是j的父结点($i, j \leq 100$)。

输出：

第一行：树的数量。

第二行：依次输出森林中树的根结点编号（从小到大）。

样例输入：

9 7

1 2

2 3

4 6

4 5

7 8

9 1

9 4

输出：

2

7 9



应用



```
int father[100+10]; //f[i]: 记录结点i的父亲
```

```
int k,m,n,t,x,y;
```

```
void main()
```

```
{
```

```
    scanf("%d%d",&n,&k);
```

```
    for(int i=1;i<=n;i++) father[i]=0;
```

```
    //默认根标志是本身，各自是独立的  
    一棵树
```



```
        for (int i=1;i<=k;i++)  
        {  
            scanf("%d%d",&x,&y);  
            father[y] =x;  
        }  
        t=0;  
        for (int i=1;i<=n;i++)  
            if (f[i]==0) t++; //统计根的个  
数  
        printf("%d\n",t);  
        for (int i=1;i<=n;i++)  
            if (f[i]==0) printf("%d ",f[i]);
```



应用



2、找树根和孩子 (treea.cpp) P366
给定一棵树，输出树的根root，孩子最多的结点max以及他的孩子。

输入：

第一行：n（结点个数），m（边数）。

以下m行；每行两个结点x和y，表示y是x的孩子。

输出：

第一行：树根：root。

第二行：孩子最多的结点max。

第三行：max的孩子。

样例
输入：

8 7

4 1

4 2

1 3

1 5

2 6

2 7

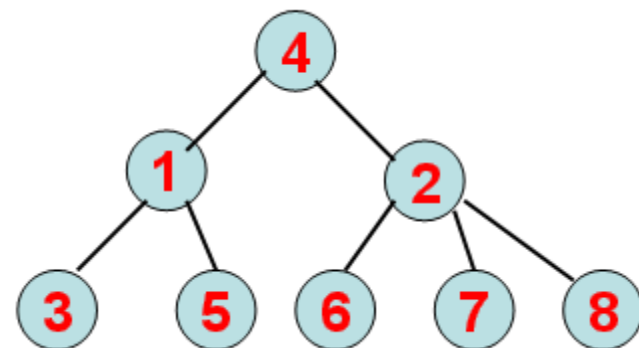
2 8

样例
输出：

4

2

6 7 8



二叉树

Please enter the title

03



二叉树的概念

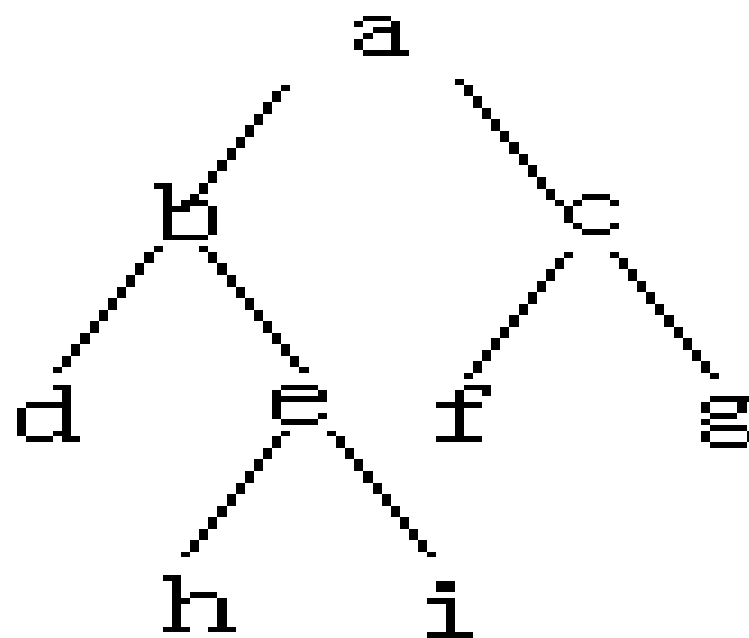


二叉树的特点:是每个结点最多有两个孩子,且其子树有左右之分 (有序树, 次序不能任意颠倒) 。

1、二叉树的递归定义和基本形态

二叉树是以结点为元素的有限集, 它或者为空, 或者满足以下条件:

- (1)有一个特定的结点称为根;**
- (2)余下的结点分为互不相交的子集L和R, 其中L是根的左子树; R是根的右子树; L和R又是二叉树;**





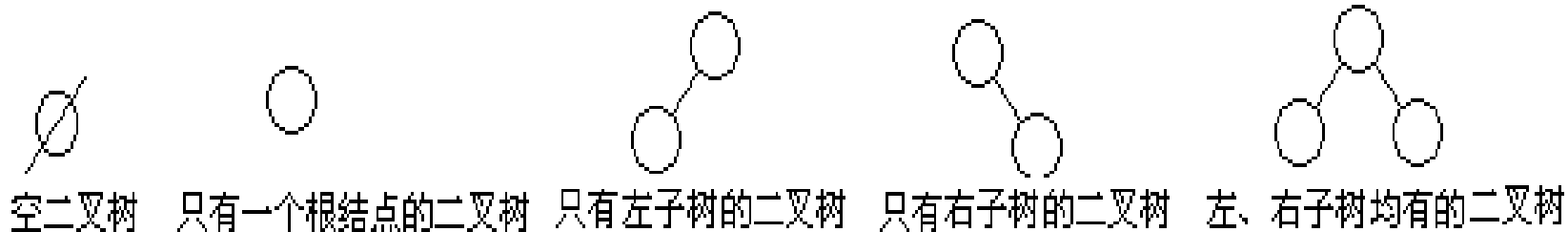
二叉树的性质



二叉树和树区别：

- (1)树的每一个结点可以有任意多个孩子，而二叉树中每个结点的孩子数不能超过2；
- (2)树的子树可以不分次序（除有序树外）；而二叉树的子树有左右之分。左儿子和右儿子。

下图列出二叉树的五种基本形态：





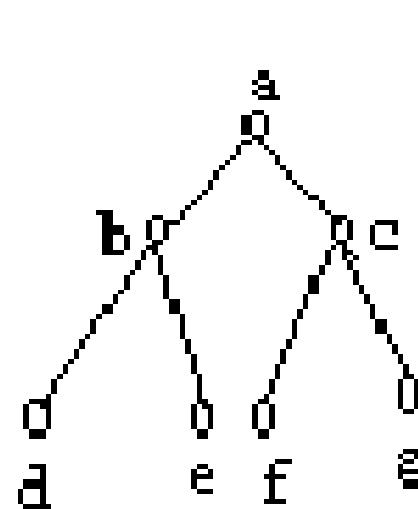
二叉树的性质



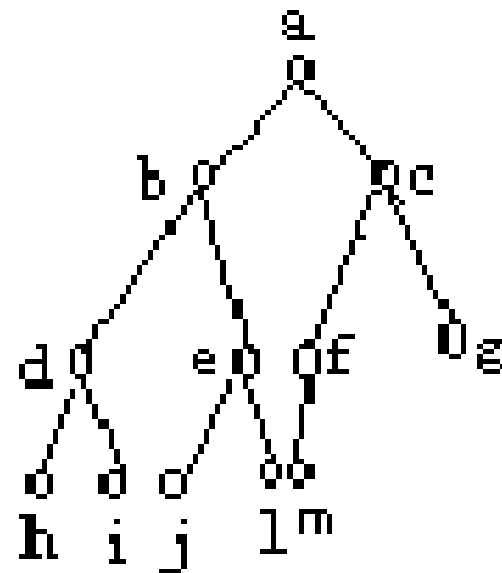
2、二叉树的两个特殊形态

(1)满二叉树： 如果一棵深度为K的二叉树，共有 2^K-1 个结点，即第I层有 2^{I-1} 的结点，称为满二叉树。(a)

(2)完全二叉树： 如果一棵二叉树最多只有最下面两层结点度数可以小于2，并且最下面一层的结点都集中在该层最左边的若干位置上，则称此二叉树为完全二叉树（例如下图(b)）



(a)



(b)



二叉树的性质

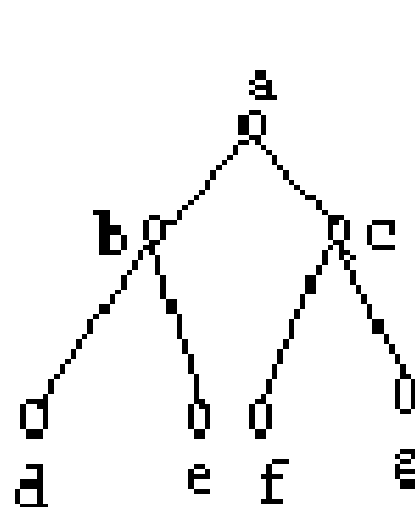


3、二叉树的主要性质

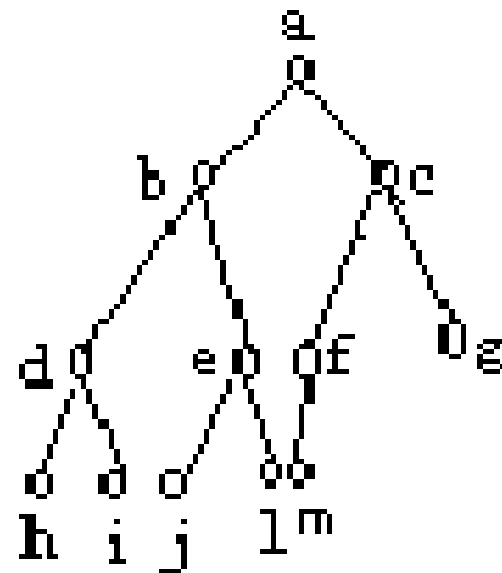
- 性质1：在二叉树的第 i (≥ 1) 层上，最多有 2^{i-1} 个结点
- 性质2：在深度为 k ($k \geq 1$) 的二叉树中最多有 $2^k - 1$ 个结点。
- 性质3：在任何二叉树中，叶子结点数总比度为2的结点多1。

$$n_0 = n_2 + 1$$

(设 n_0 为二叉树的叶结点数； n_2 为二叉树中度为2的结点数)



(a)



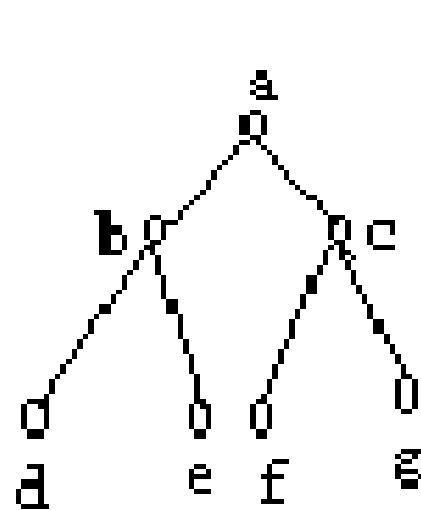
(b)



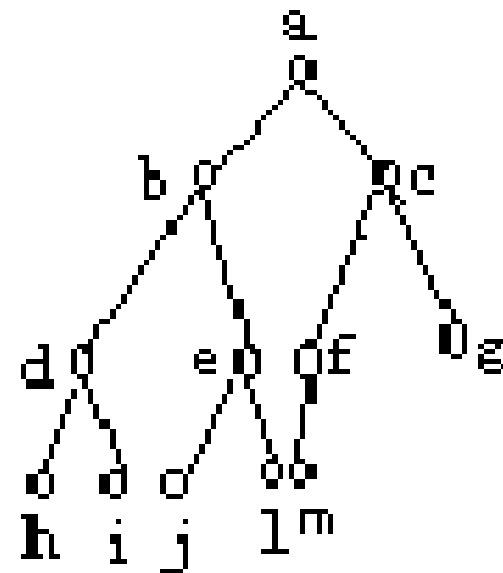
二叉树的性质



- 性质4：具有 n 个结点的完全二叉树的深度为 $\text{floor}(\log_2 n) + 1$
- 性质5：对于一棵 n 个结点的完全二叉树，对任一结点（编号 i ）有：
 - 1 如果 $i=1$,则结点 i 为根，无父结点；如果 $i>1$ 则其父结点编号为 $i/2$
 - 2 如果 $2*i>n$,则结点 i 无左孩子（当然也无右孩子），即为叶子结点，否则左孩子编号为 $2*i$ 。
 - 3 如果 $2*i+1>n$ ，则结点 i 无右孩子，否则右孩子编号为 $2*i+1$ 。



(a)



(b)



二叉树的性质



设 n_0 为二叉树的叶结点数； n_1 为二叉树中度为1的结点数； n_2 为二叉树中度为2的结点数，显然

$$n = n_0 + n_1 + n_2 \quad (1)$$

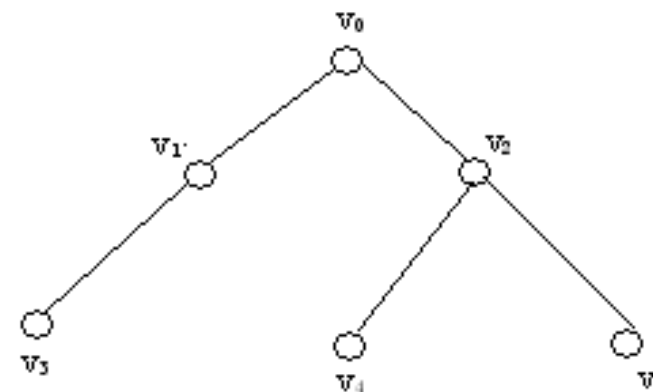
由于二叉树中除了根结点外，其余每个结点都有且仅有一个边指向父亲。设 b 为二叉树的边的个数，

$$n = b + 1 \quad (2)$$

所有这些分支同时又为度为1和度为2的结点发出的。因此又有

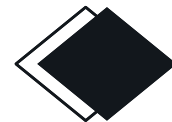
$$b = n_1 + 2n_2 \quad (3)$$

由1, 2, 3得到： $n_0 = n_2 + 1$





二叉树的性质



[试题分析]：如果一棵 m 度的树中有 N_1 个度为1的顶点， N_2 个度为2的顶点， N_3 个度为3的顶点，……， N_m 个度为 m 的顶点，求该树中叶子顶点个数。

分析：设叶子结点数为 N_0

所有结点数为 n ，边数(分支)为 b ，则有：

$$n = b + 1 \quad (1)$$

$$\text{又： } n = N_0 + N_1 + N_2 + \dots + N_M \quad (2)$$

$$b = N_1 + 2N_2 + 3N_3 + \dots + M \cdot N_M \quad (3)$$

$$(2), (3) \text{ 代入 } (1) \text{ 得出： } N_0 = N_2 + 2N_3 + 3N_4 + \dots + (M-1)N_M + 1$$



二叉树的性质



①(NOIP9)一个高度为 h 的二叉树最小元素数目 ()。

A) $2h+1$ B) h C) $2h-1$ D) $2h$ E) $2h-1$

②(NOIP8)按照二叉数的定义, 具有3个结点的二叉树有 () 种。

A) 3 B) 4 C) 5 D) 6

③(NOIP8)设有一棵 k 叉树, 其中只有度为0和 k 两种结点, 设 n_0 , n_k , 分别表示度为0和度为 k 的结点个数, 试求出 n_0 和 n_k 之间的关系 ($n_0 =$ 数学表达式, 数学表达式仅含 n_k 、 k 和数字)。

④(NOIP7).一棵二叉树的高度为 h , 所有结点的度为0, 或为2, 则此树最少有()个结点

A) 2^h-1 B) $2h-1$ C) $2h+1$ D) $h+1$

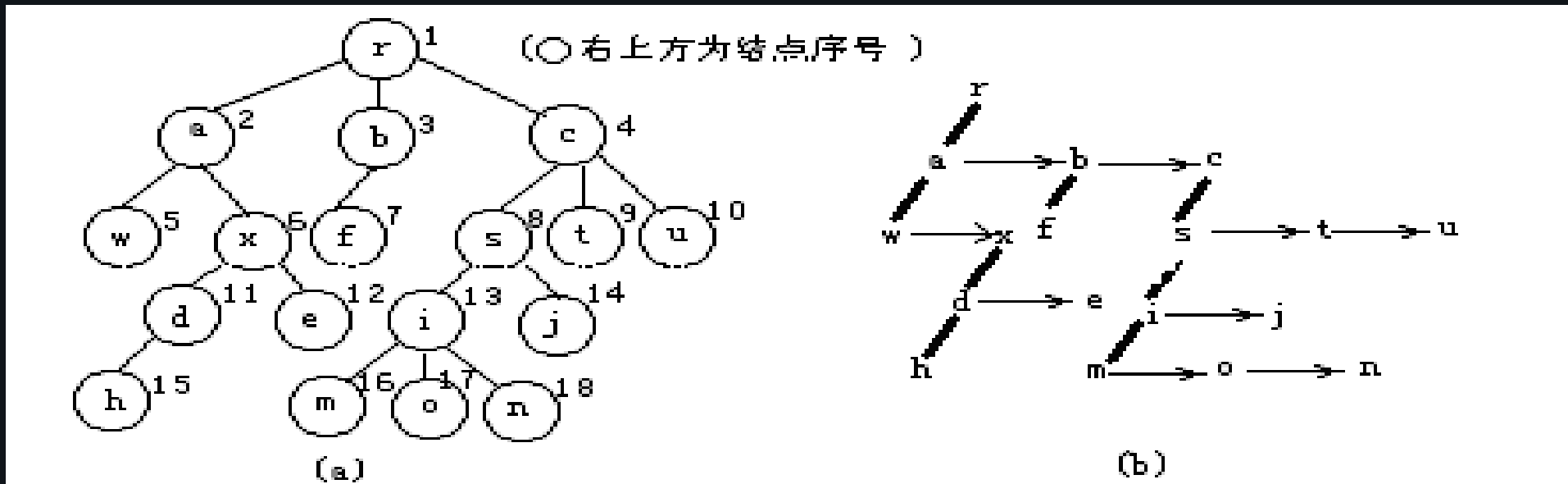


树与二叉树转换



1、树转化为二叉树(左孩子右兄弟)

一棵有序树转化成二叉树的根结点是没有右子树的，并且除保留每个结点的最左分支外，其余分支应去掉，然后从最左的儿子开始沿右儿子方向依次链接该结点的全部儿子。例如将下图 (a) 所示的普通有序树转换成二叉树(下图 (b))。



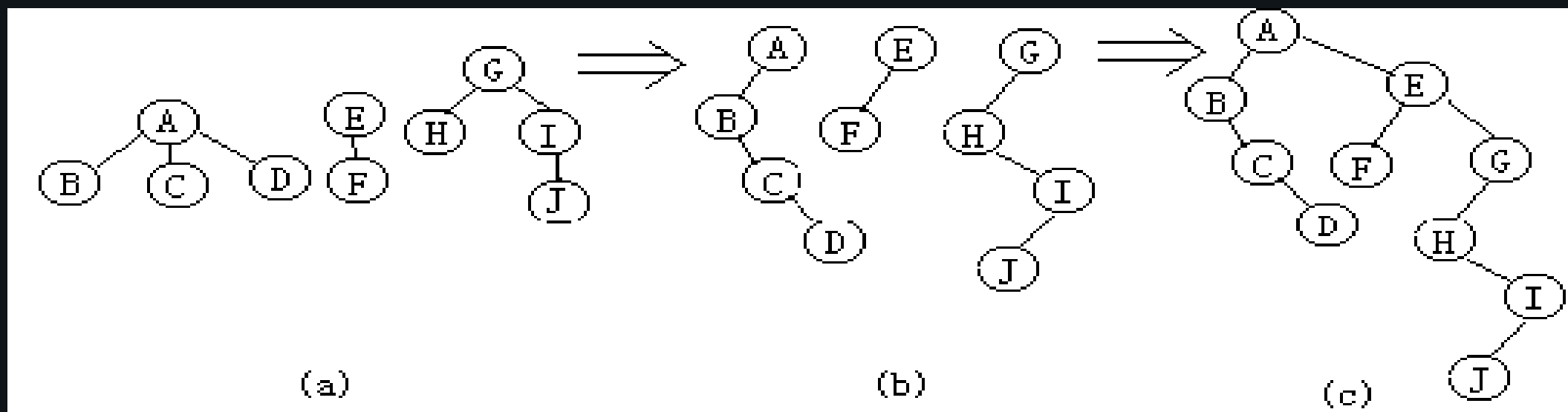


树与二叉树转换



2、森林转换成二叉树

- (1)将森林中的每棵树转换成相应的二叉树;
- (2)将各棵树的根相连;
- (3)以第一棵树为轴心, 顺时针粗略地旋转900;





二叉树存储



1、顺序存储结构

将每个结点依次存放在一维数组中，用数组下标指示结点编号，编号的方法是从根结点开始编号1，然后由左而右进行连续编号。每个结点的信息包括

```
const int maxn=树中结点数上限;
```

```
typedef struct node; //结构体类型声明
```

```
struct node
```

```
{
```

```
    int data; //数据值
```

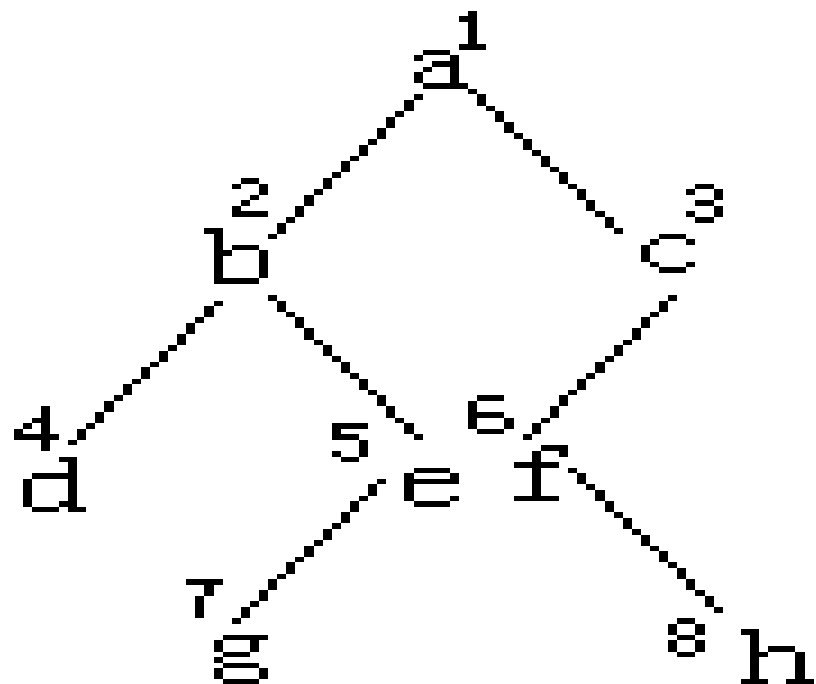
```
    int father, lch, rch //父结点、左儿子、右儿子编号
```

```
}
```

```
node tree[maxn]; //二叉树的顺序表类型
```



二叉树的存储



(a)

	data	pri	lch	rch
1	a	0	2	3
2	b	1	4	5
3	c	1	6	0
4	d	2	0	0
5	e	2	7	0
6	f	3	0	8
7	g	5	0	0
8	h	6	0	0

(b)

例如，采用数组tree存储二叉树（如图）



二叉树的遍历



六、二叉树的遍历(访问)

所谓二叉树的遍历是指按照一定的规律不重复地访问二叉树中的每一个结点。

如果用L、D、R分别表示遍历^左子树、访问^根结点、遍历^右子树，则对二叉树的遍历可以有列六种 ($3!=6$) 组合：

LDR、LRD、DLR、DRL、RDL、RLD

若再限定^{先左后右}的次序，则只剩下三种组合

DLR、LDR、LRD

这三种遍历规则分别称为先（前）序遍历、中序遍历和后序遍历（^{以根为标准}）。



二叉树的遍历



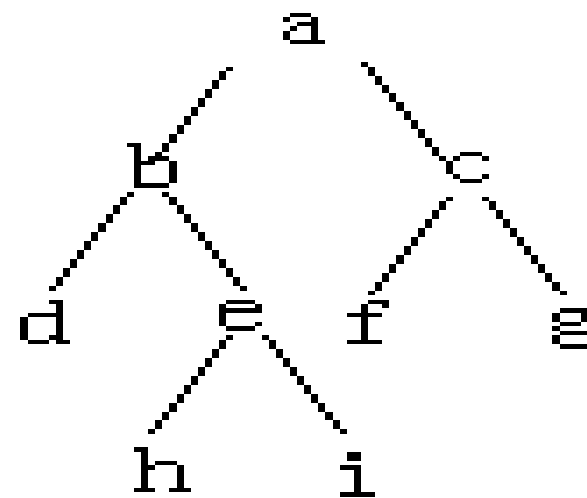
(1)、前（根）序遍历

前序遍历的规则如下：

若二叉树为空，则退出。否则

- (1)访问处理根结点；
- (2)前序遍历左子树；
- (3)前序遍历右子树；

a b d e h i c f g





二叉树的遍历



(2)中序遍历

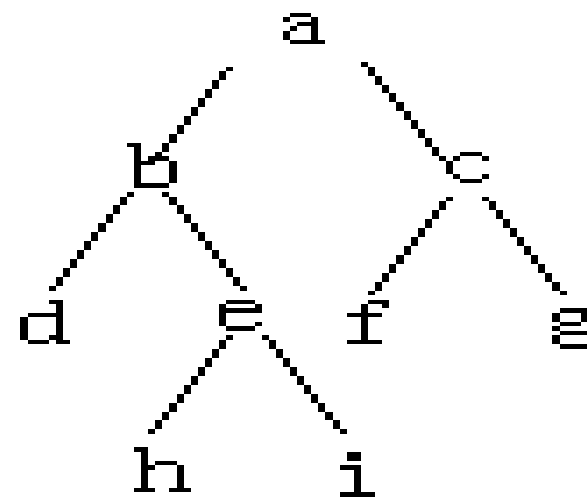
中序遍历的规则如下：

若二叉树为空，则退出；否则

- (1)中序遍历左子树；
- (2)访问处理根结点；
- (3)中序遍历右子树；

若中序遍历上图中的二叉树，可以得到如下的中序序列：

d b h e i a f c g





二叉树的遍历



(3)后序遍历

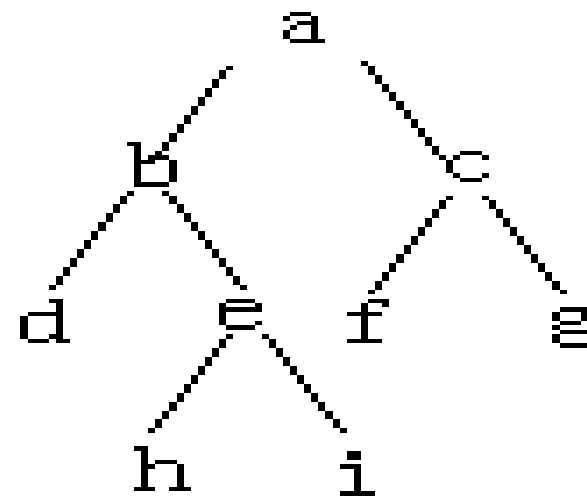
后序遍历的规则如下：

若二叉树为空，则退出；否则

- (1)后序遍历左子树；
- (2)后序遍历右子树；
- (3)访问处理根结点；

若后序遍历上图中的二叉树，可以得到如下的后序序列

d h i e b f g c a





二叉树的遍历



1、将一棵有100个结点的完全二叉树从根结点这一层开始，每一层从左到右依次对结点进行编号，根结点的编号为1，则编号为49的结点的左孩子的编号为[]

A 98 B 99 C 97 D 50

2、对二叉树从1进行连续编号，要求每个结点的编号大于其左右孩子的编号，同一结点的左右孩子中，其左孩子的编号小于其右孩子的编号，则可以采取[]次序的遍历方法。

A 先序 B中序 C后序 D从根开始的层次遍历

3、有n个结点并且其高度为n的二叉树的数目是[]

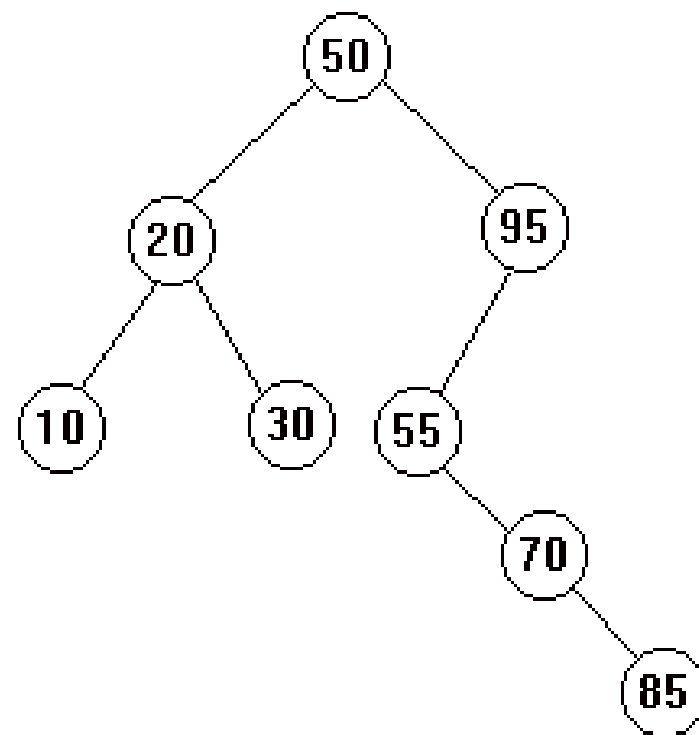
A、 n B、 $2n$ C、 2^{n-1} D、 $2(n-1)$



二叉树的遍历



4. 写出二叉树三种序遍历结果





二叉树的遍历



1、编程实现：二叉树的遍历 (tree1.cpp)

建立二叉树，然后实现：输出先序遍历、中序遍历、后序遍历的结果。

输入：

第一行：结点个数n。

以下行：每行3个数，第一个是父亲，后两个依次为左右孩子，0表示空。

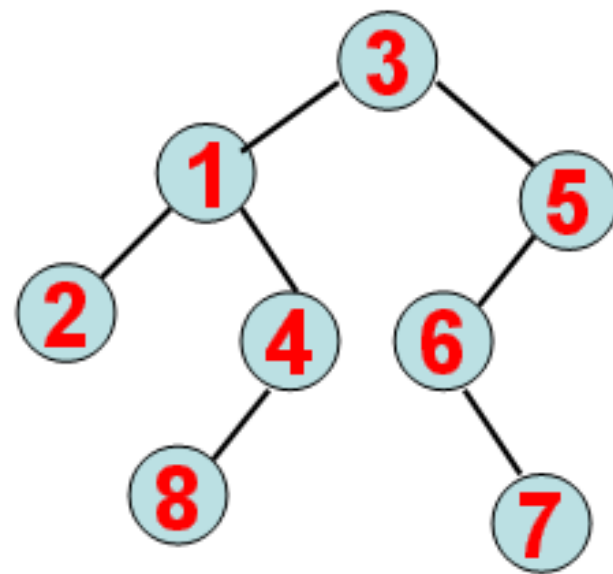
输出：根、先中后序遍历结果。

样例输入：

```
8
1 2 4
2 0 0
4 8 0
3 1 5
5 6 0
6 0 7
8 0 0
7 0 0
```

样例输出：

```
3
3 1 2 4 8 5 6 7
2 1 8 4 3 6 7 5
2 8 4 1 7 6 5 3
```



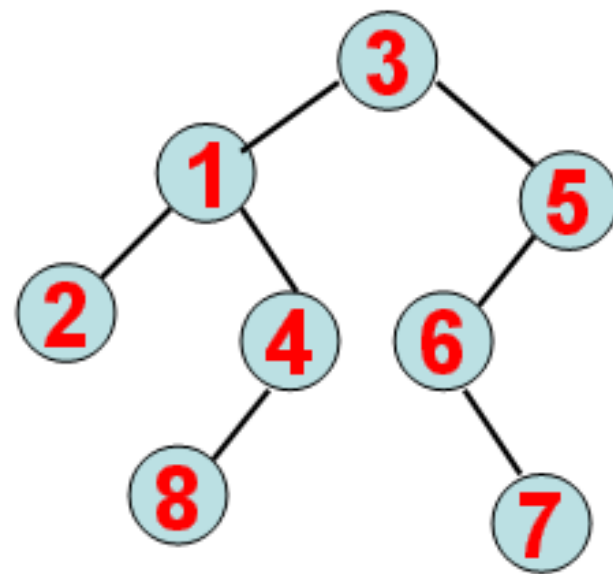


二叉树的遍历



遍历算法的简单变化:

- 1) 设计算法按照从左向右的顺序输出二叉树的叶子结点。
- 2) 设计算法求二叉树中度为2的结点数。
- 3) 设计算法求二叉树的深度。





二叉树的遍历



七、由二叉树的两种遍历顺序确定树结构

遍历二叉树（如下图）有三种规则：

前序遍历：根—左子树—右子树；

中序遍历：左子树—根—右子树；

后序遍历：左子树—右子树—根；

1、已知

先序遍历结果：ABCDEFGH

中序遍历结果：CBEDAGHF

求后序遍历结果。

2、(NOIP7)已知

中序：CBGEAFHDIJ

后序：CGEBHFJIDA

该二叉树的先序遍历的顺序为：



二叉树的遍历



1、已知

先序遍历结果: ABCDEFGH

中序遍历结果: CBEDAGHF

求后序遍历结果。

- 首先以先序遍历的首字母，遍历中序序列，中找到该字母，然后以该字母为分界线，把序列分成左右两个部分，分别建左右两个子树，再对子序列这样子进行，直到只剩下一个字母为止。

```
void dfs(int l,int r,int k)
{
    if(l>=r)
    {
        if(l==r)printf("%c",b[r]);
        return ;
    }
    int m;
    for(int i=l;i<=r;i++)
        if(a[k]==b[i])
        {
            m=i;
            break;
        }
    dfs(l,m-1,k+1);
    dfs(m+1,r,k+m-l+1);
    printf("%c",b[m]);
}
```



二叉树的遍历



1、树的公共祖先：

给定一棵二叉树和两个不同的节点，求出他们最近的公共祖先父节点。已知该二叉树有 n 个节点，标号 $1..n$ 。 $(n < 100)$

输入：

第一行三个整数 n ， x ， y ，表示需要计算的节点；

以下若干行，每行两个整数 a 和 b ，表示 a 的父节点是 b 。

输出：

x 与 y 的最近公共祖先 $root$ 。

输入样例：

9 9 7

2 1

3 2

4 2

5 3

8 5

9 5

6 4

7 4

输出样例：

2



二叉树的遍历



先从节点x开始向根遍历，然后做上标记，再从节点y开始向根遍历，第一次遇到已访问过的节点，则结束。

- 时间复杂度： $2*n$

问题扩展：

- 1、如果有多个询问呢？
- 2、如果n和m很大呢？ (n,m=100000)

输入样例：

9 9 7

2 1

3 2

4 2

5 3

8 5

9 5

6 4

7 4

输出样例：

2



二叉树的遍历



2、树的路径

给定一棵二叉树和两个不同的节点，求出两个节点的路径。已知该二叉树有 n 个节点，标号 $1..n$ 。
($n < 100$)

输入：

第一行两个整数 x ， y ，表示需要求的节点；

以下若干行，每行两个整数 a 和 b ，表示 a 的父节点是 b 。

输出：

x 到 y 的路径。

输入样例：

9 7

2 1

3 2

4 2

5 3

8 5

9 5

6 4

7 4

输出样例：

9 5 3 2 4 7



2019

THANK YOU